

ОТЧЁТ О ВЫПОЛНЕНИИ ИНДИВИДУАЛЬНОГО ПРОЕКТА

Дисциплина: «Распределённые и облачные базы данных»

Поле	Значение
ФИО	Ермаков Лаврентий
Группа	М09-КИИ26
Дата	18.09.2026
Каталог	Индивидуальный_проект/

1. Описание ТЗ и связь с темой работы

Тема проекта: «Горизонтально фрагментированная СУБД контура исполнения отгрузок на распределённых складских площадках».

Предметная область — операционный контур площадок: остатки, резерв, сборка, отгрузка, подтверждение доставки. Это не интернет-магазин и не витрина для покупателя. Учебное упрощение контура: два региона (WEST/EAST), синтетические данные, имитация площадок контейнерами Docker; критерий распределения и смысл операций сохранены.

Требования задания, закрытые в проекте:

- распределённая БД с явным критерием размещения (не «балансировка»);
- инфологическая и даталогическая модели в ЗНФ с внешними ключами;
- стенд ≥ 2 процессов PostgreSQL (реализовано 4: 2 региона $\times$ primary + replica);
- клиент: CRUD ключевых сущностей + учёт распределения + предметный сценарий сверх CRUD;
- отказ узла, чтение с реплики, отказ при нарушении критерия контура;
- электронно: docker-compose, SQL/seed, клиент, зависимости, скрипты демо.

1.1. Контур данных

Таблица	По-русски	Назначение
regions	регионы	Периметр площадки (WEST/EAST)
warehouses	склады	Склад внутри региона
products	товары	Справочник номенклатуры (реплика на шардах)
stock_balances	остатки	qty_available / qty_reserved
shipment_orders	заявки	Наряд: склад, region_code, ТЕКУЩИЙ status
shipment_items	позиции	Товар и количество в заявке (order_id $\rightarrow$ orders.id)
shipment_events	журнал	История смен статусов: from $\rightarrow$ to, время, note

Журнал vs статус заявки. Поле shipment_orders.status хранит только текущий статус. Таблица shipment_events — это журнал: каждая строка = один переход (from_status $\rightarrow$ to_status), плюс created_at, note, order_id (= shipment_orders.id). Примеры: NULL $\rightarrow$ CREATED, CREATED $\rightarrow$ RESERVED, ... $\rightarrow$ DELIVERED / CANCELLED. Журнал лежит на том же фрагменте, что и заявка (производная фрагментация).

1.1.1. Как работает журнал (подробно)

Журнал фиксирует каждое действие смены статуса заявки в пределах своего фрагмента (WEST — только заявки WEST, EAST — только EAST). Это не общий лог на всю систему и не «лог сервера PostgreSQL», а предметная таблица истории исполнения наряда.

Поле строки events	Что означает
order_id	Какая заявка (FK $\rightarrow$ shipment_orders.id)
from_status	Какой статус был до перехода (для создания заявки — NULL)
to_status	Какой статус стал после перехода

Поле строки events	Что означает
created_at	Когда зафиксировали переход
note	Комментарий / зачем сменили (контроль)

Запись журнала выполняется только на primary региона вместе со сменой статуса (один и тот же запрос к primary: UPDATE orders + INSERT events). Потом async replication копирует новые строки events на replica.

Чтение журнала (GET .../orders/{id}/events) идёт предпочтительно с replica того же региона; если replica недоступна — клиент читает с primary. Одновременно с primary и replica клиент не читает: выбирается один узел. Небольшой лаг replica возможен сразу после записи.

CREATE → INSERT event (NULL → CREATED) // на primary
 RESERVED → INSERT event (CREATED → RESERVED) // на primary
 PICKING / SHIPPED / DELIVERED — тоже по одной строке events
 GET events → SELECT с replica (fallback primary) // только чтение

1.2. Профиль запросов

Где	Операции
Узел региона (площадка)	CRUD остатков/складов; создание заявки; резерв; статусы; события
Replica региона	Только чтение остатков, заявок, событий
Межрегионно / «центр»	Справочник products; запрещены кросс-региональные UPDATE остатков
Несколько узлов	Только при ошибочном запросе чужого региона → отказ по критерию

1.3. Безопасность и надёжность (контекст объекта КИИ)

Профиль направления — объекты критической информационной инфраструктуры (КИИ). В учебном стенде это отражено не «магазином с паролями», а правилами размещения и отказоустойчивости оперативных данных обеспечения площадок.

Безопасность (режим доступа / периметр)

- Оперативка региона не покидает периметр своего фрагмента: строка с region_code=WEST существует только на узлах WEST.
- Клиент отказывает в записи/смене статуса чужого региона (HTTP 403), даже если другой шард жив — это контроль нарушения контура, а не «фолбэк на соседний сервер».
- Роли узлов разделены: на replica писать нельзя (hot standby read-only); приложение направляет запись только на primary.
- Журнал shipment_events фиксирует каждую смену статуса (аналог контроля/квотирования) — кто/когда перевёл заявку, для аудита исполнения.
- Учётные данные стенда учебные; в отчёт боевые секреты не включаются (требование задания).

Надёжность (доступность при отказах)

- Внутри региона — primary + async replica: при падении primary чтение продолжается с replica; при падении replica запись на primary сохраняется.
- Автономность площадки: потеря канала до другого региона не останавливает локальные отгрузки на своём primary.
- Другой регион — другой фрагмент, а не «запасная копия для балансировки»: гибель WEST не подменяет данные EAST и наоборот (нет скрытого смещения периметров).
- Цепочка статусов с проверками остатка (нельзя зарезервировать больше available) снижает риск некорректного исполнения наряда при сбоях/повторных запросах.
- Модель отказа и демо docker stop primary описаны в §5; сбор глобальной картины без нарушения периметра — в §3.2.

Итого для КИИ-логики стенда: безопасность = резидентность оперативки и запрет кросс-контурных операций; надёжность = реплика на чтение, автономность региона и предсказуемое поведение при отказе узла. Полный контур аттестации КИИ (СЗИ, криптография каналов, ИБ-политики) в учебном проекте намеренно упрощён — сохранены критерий и смысл операций.

2. UML структуры данных (ER)

Модель в третьей нормальной форме: связи через FK, без дублирования названия региона в каждой строке остатка.

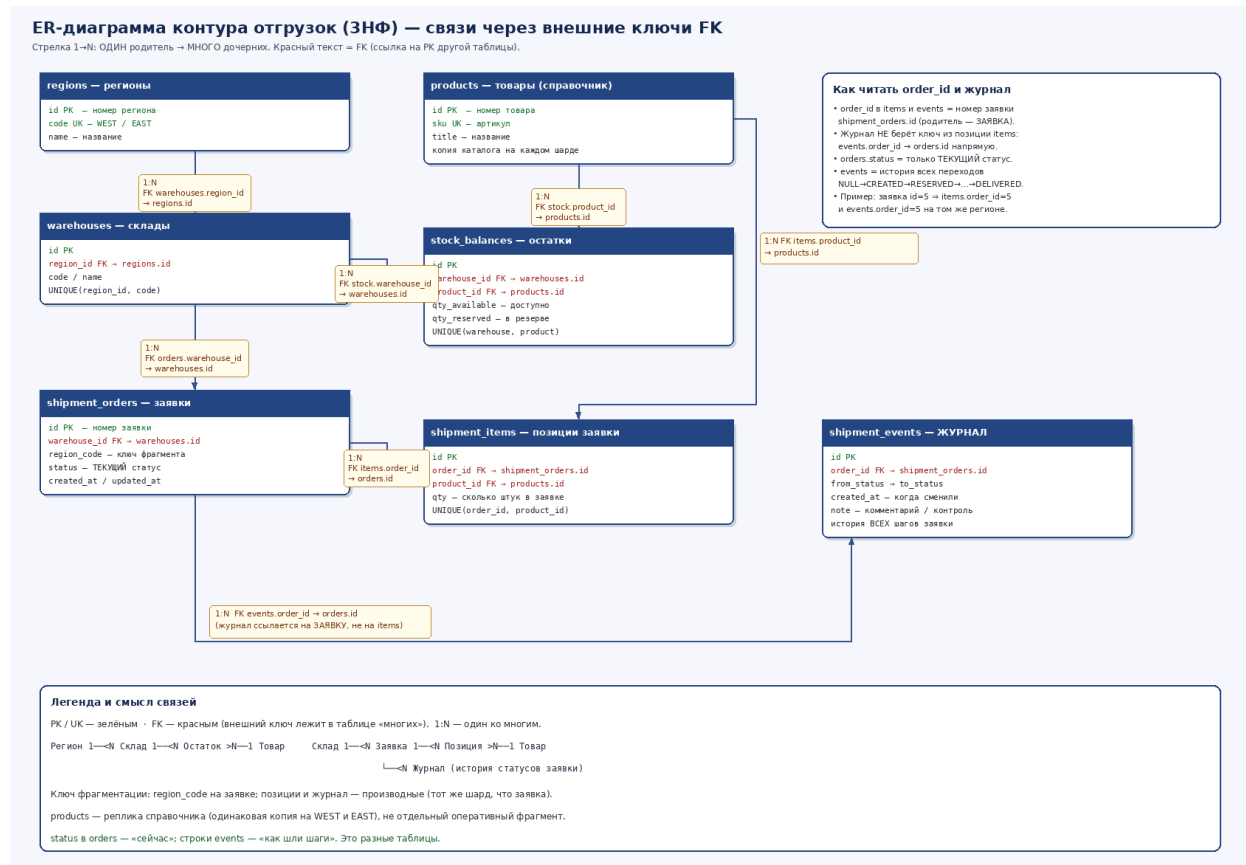


Рисунок 1 — UML / ER: структура данных (ЗНФ, PK/FK, 1:N)

Region 1—<N Warehouse 1—<N StockBalance >N—1 Product
 |
 |—<N ShipmentOrder 1—<N ShipmentItem >N—1 Product
 |
 |—<N ShipmentEvent

2.1. UML sequence — сценарий отгрузки WEST + отказы

Диаграмма последовательностей показывает, как клиент взаимодействует с роутером и узлами: запись идёт на primary, чтение событий — с replica, попытка трогать чужой регион даёт отказ по критерию, при падении primary запись запрещена, а чтение остаётся через replica.

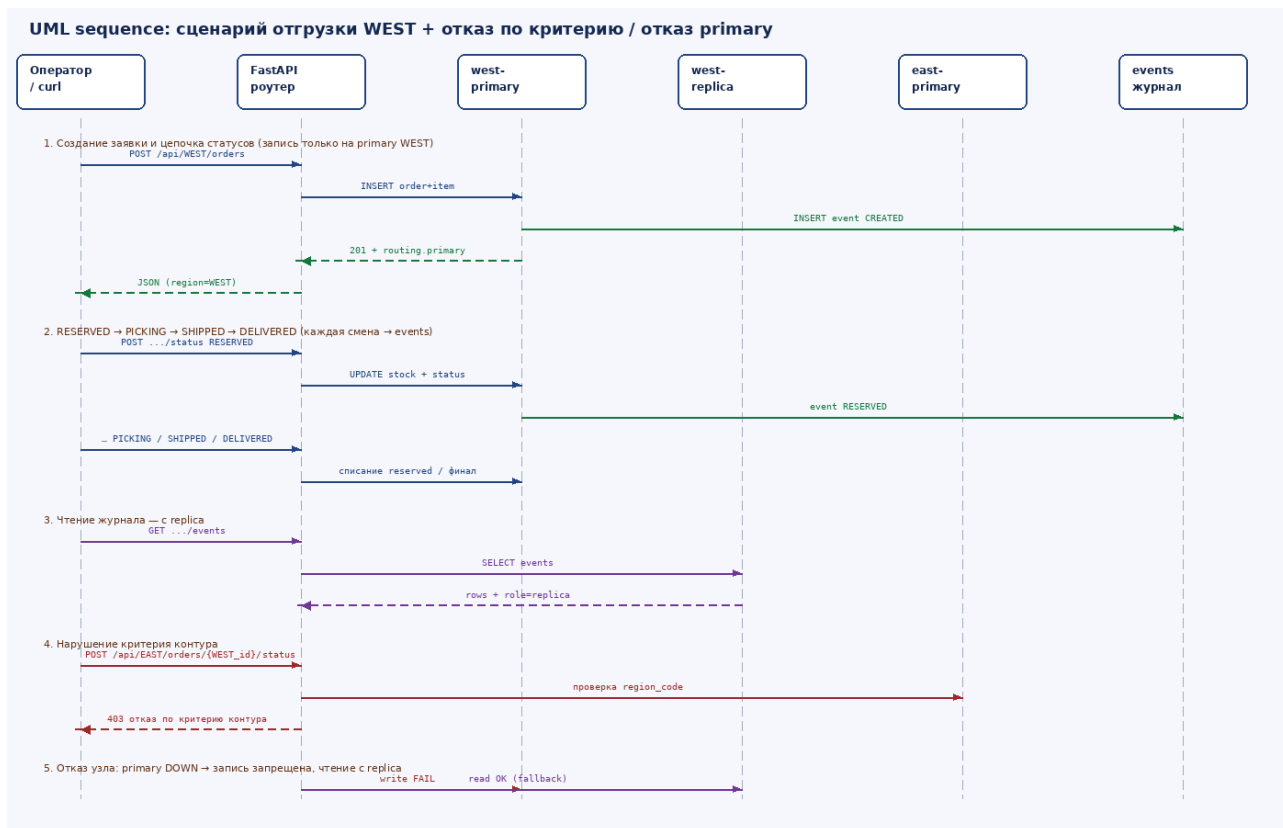


Рисунок 1a — UML sequence: отгрузка WEST + отказ контура / primary

3. Схема распределения и критерий

Критерий: по контуру / площадке + по режиму доступа. Ключ размещения — `region_code`. Оперативка региона существует только на узлах этого региона. Ответ «распределили для балансировки» не используется.

3.0. Что означают термины фрагментации и репликации

Ниже — те же понятия простыми словами (как отвечать на защите).

- Горизонтальная первичная фрагментация. Таблица «режется по строкам», а не по столбцам. Правило: все строки оперативки с `region_code = WEST` лежат только на узлах WEST; с EAST — только на EAST. «Первичная» значит: это основной разрез предметных таблиц (`stock_balances`, `shipment_orders`), от которого дальше зависят связанные таблицы.
- Производная фрагментация (`items / events`). `shipment_items` и `shipment_events` сами по себе не режутся отдельным ключом: они следуют за заявкой. Если заявка WEST, то её позиции и журнал событий тоже только на WEST. Иначе связь `order→items` разъехалась бы по разным серверам.
- Async streaming primary → replica. Внутри региона PostgreSQL передаёт журнал WAL с primary на replica потоком и асинхронно: запись подтверждается на primary, реплика догоняет с лагом в секунды. Replica — только чтение (`hot standby`). Это не второй фрагмент данных, а копия того же фрагмента для надёжности чтения.
- `products` — реплика справочника. Номенклатура нужна и WEST, и EAST, но это не «ещё один кусок оперативки». На оба шарда кладётся одинаковая копия справочника (при `bootstrap/seed`). Менять остатки чужого региона через `products` нельзя: `products` не содержит `qty` и не является шардом оперативного контура.

Параметр	Значение
Фрагментация	горизонтальная первичная по <code>region_code</code>
Производная	<code>shipment_items / shipment_events</code> следуют за заявкой
Репликация	async streaming WAL: primary → hot standby внутри региона
Справочник	<code>products</code> — реплика справочника на обоих шардах

3.0.1. Полнота, непересечение, восстановимость — явно

Это три условия корректности горизонтальной фрагментации (проверка, что разрез сделан правильно, а не «просто два сервера»).

Условие	Что это значит	Как у нас выполняется
Полнота	Если сложить все фрагменты, получится весь набор данных контура — ничего не «потерялось между серверами».	WEST и EAST = все склады, остатки и заявки стенда. Нет третьего скрытого шарда.
Непересечение	Одна и та же оперативная строка не лежит сразу на двух фрагментах (нет дублей «и WEST, и EAST»).	Заявка/остаток с <code>region_code=X</code> существует только на узлах X. Клиент запрещает писать чужой регион (403).
Восстановимость	По фрагментам можно снова понять глобальную картину без потери смысла (собрать ответ «по всей системе»).	Опрос GET WEST + GET EAST на стороне клиента + общий справочник <code>products</code> . Межшардовый JOIN оперативки не нужен.

Кратко на защите: полнота — «всё на месте в сумме»; непересечение — «нет двойных копий оперативки»; восстановимость — «можем снова собрать общую картину, опросив фрагменты».

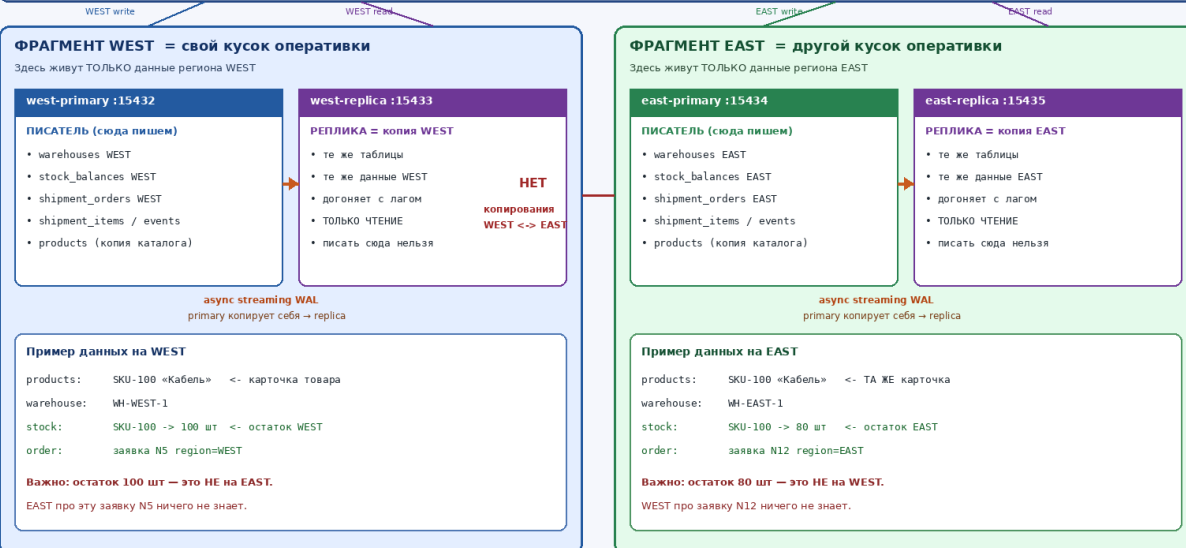
Как устроен стенд: WEST / EAST, фрагменты, реплики, товары

Два региона = два разных куска оперативки; у каждого своя копия для чтения; товары = общий каталог (две одинаковые копии)

КЛИЕНТ FastAPI :8000 (Swagger <http://127.0.0.1:8000/docs>)

URL решает регион: /api/WEST/... или /api/EAST/...

запись → primary выбранного региона | чтение → replica | чужой регион → отказ 403



ГЛАВНАЯ МЫСЛЬ ПРО ТОВАРЫ

products = справочник (карточка «что такое SKU-100»). Он НЕ делится на west-товар / east-товар.

На WEST и EAST лежат ДВЕ ОДИНАКОВЫЕ КОПИИ каталога (реплика справочника), не одна общая физическая таблица.

Остатки (stock) = сколько лежит на складе региона — они РАЗНЫЕ: WEST 100 шт, EAST 80 шт одного и того же SKU-100.

Заявки тоже свои у региона. Попытка менять заявку WEST через /api/EAST/... → отказ 403 (периметр).

Фрагмент = свой набор строк оперативки. Реплика = копия СВОЕГО фрагмента внутри региона (не копия чужого региона).

Рисунок 2 — схема распределения: фрагменты WEST/EAST, primary/replica, справочник products

По смыслу это та же схема «фрагмент + реплика», что и UML deployment: два независимых куска оперативки, запись на primary, чтение с replica, между WEST и EAST копирования оперативки нет; products — одинаковая копия каталога на обоих.

3.1. Фрагмент vs реплика

Объект	Классификация
west-primary / east-primary	фрагмент оперативных данных своего региона
west-replica / east-replica	реплика своего фрагмента (только чтение)
копия products на шардах	реплика справочника, не отдельный оперативный фрагмент

3.2. Как данные «собираются воедино» (глобальная картина)

Оперативные данные не сливаются в одну физическую БД и не требуют межшардового JOIN остатков/заявок. Глобальная картина получается логически за счёт трёх условий выше:

- Полнота на практике. Объединение ответов WEST и EAST даёт весь контур.
- Непересечение на практике. В склейке нет двух одинаковых order_id из разных регионов как «одной» заявки — у каждого свой периметр.
- Восстановимость на практике. Клиент вызывает GET /api/WEST/orders и GET /api/EAST/orders и склеивает списки; справочник products уже согласован на обоих шардах.
- Что не делается. Нет единого центрального primary для всей оперативки и нет FDW-JOIN остатков WEST↔EAST — иначе нарушился бы периметр.

Внутри региона согласованность primary и replica даёт async streaming: после записи на primary реплика догоняет за секунды (допустимый лаг).

4. Техническая реализация стенда

Стенд: Docker Compose, четыре независимых процесса PostgreSQL 16. Техника: шардинг по region (клиентский роутинг) + асинхронная потоковая репликация primary→replica внутри региона. Имитация площадок контейнерами допускается заданием.

Узел	Порт	Роль	Фрагмент	Кто пишет	Канал	Лог
west-primary	15432	писатель	WEST	клиент	Docker net	—
west-replica	15433	читатель	реплика WEST	никто	streaming async	секунды
east-primary	15434	писатель	EAST	клиент	Docker net	—
east-replica	15435	читатель	реплика EAST	никто	streaming async	секунды
FastAPI	8000	роутер	—	—	localhost	—

Артефакты поставки:

- docker-compose.yml — 4 узла + healthcheck;
- db/init/01_schema.sql + seed WEST/EAST;
- db/primary/00_replication.sh, db/replica/entrypoint.sh;
- client/app/main.py + requirements.txt;
- scripts/bootstrap_schema.sh, demo.sh, захват скриншотов.

4.1. Подъём стенда (кратко)

```
unset COMPOSE_PROJECT_NAME
docker compose up -d
source .venv/bin/activate && pip install -r client/requirements.txt
./scripts/bootstrap_schema.sh
uvicorn client.app.main:app --port 8000
./scripts/demo.sh
```

5. Модель отказа узла

Ниже — ответ на вопрос задания: что будет, если один из серверов «умрёт». В стенде «сервер» = контейнер PostgreSQL (primary или replica региона).

Событие	Поведение
Отказ primary региона	Запись в регион запрещена; чтение — с replica; другой регион работает
Отказ replica	Запись на primary сохраняется; чтение переключается на primary
Потеря канала до чужого региона	Локальные операции на своём primary продолжаются
Запись / смена статуса чужого региона	Отказ по критерию контура (даже если другой узел жив)
Восстановление primary	docker start; запись возобновляется после healthy

5.1. Разбор по ролям узлов

- Умер west-primary. Клиент не может создать заявку / сменить статус WEST (ошибка «узел недоступен»). GET остатков и events WEST всё ещё возможны с west-replica. EAST при этом работает как обычно — это другой фрагмент, не копия WEST.
- Умер west-replica. Запись на west-primary продолжается. Чтение WEST клиент переключает на primary (в ответе routing.role = primary).
- Умер весь регион WEST (оба узла). Фрагмент WEST недоступен: операции с region=WEST отклоняются. Фрагмент EAST жив. Глобальную картину «собрать» полностью нельзя, пока WEST не восстановлен — это ожидаемо при горизонтальной фрагментации (нет второй копии оперативки WEST на EAST).
- Потеря канала «между регионами». Каждый регион автономен: локальные отгрузки на своём primary идут дальше. Нет требования держать связь с чужим шардом для оперативки.
- Демо на стенде. docker stop rdb-west-primary → POST заявки WEST падает, GET stock WEST читает с replica; затем docker start rdb-west-primary — запись снова доступна (скрины в §8).

Promote replica в новый primary в учебном стенде не автоматизирован: достаточно показать отказ записи и чтение с replica, затем поднять primary обратно. В боевой схеме после гибели primary replica повышают (pg_promote) и перенастраивают клиента на новый писатель.

6. Сценарий демонстрации клиента

Клиент — веб-API на FastAPI. Интерфейс для ручной проверки — Swagger UI: откройте в браузере <http://127.0.0.1:8000/docs> (клиент должен быть запущен через uvicorn). Каждый ответ содержит блок routing: region, role (primary/replica), host — куда реально ушёл запрос.

6.0. Как пользоваться кнопками в Swagger (общий порядок)

- Слева/сверху список операций с цветными метками: GET (зелёный) — чтение, POST (синий) — создать / сменить статус, PUT (оранжевый) — изменить, DELETE (красный) — удалить.
- Нажмите на строку операции — раскроется описание и поля.
- Кнопка Try it out — включает режим ввода (иначе поля только для просмотра).
- В пути почти везде параметр region: впишите WEST или EAST (регистр не важен, клиент приведёт к верхнему).
- Если есть Request body — отредактируйте JSON-пример под свои id/значения.
- Кнопка Execute — отправить запрос. Ниже появятся Code (200/403/...) и Response body.
- Кнопка Clear / свернуть операцию — сбросить ввод. Справа от пути можно скопировать curl — то же самое из терминала.

Важно: запись всегда идёт на primary выбранного региона, чтение (GET) — предпочтительно с replica. Нельзя через WEST менять заявку EAST (будет 403).

6.0.1. Службная проверка узлов

Кнопка / метод	Что делает	Как пользоваться
GET /health	Проверяет доступность всех 4 узлов (WEST/EAST × primary/replica)	Try it out → Execute. Смотрите ok: true/false по каждому узлу. Перед демо убедитесь, что все true.

6.0.2. Склады (warehouses)

Кнопка / метод	Что делает	Как пользоваться
GET .../warehouses	Список складов региона (чтение с replica)	region=WEST или EAST → Execute. В ответе data[] с id, code, name.
POST .../warehouses	Создать склад (запись на primary)	region=WEST. Body: {"code":"WH-WEST-2","name":"Склад Запад-2"}. Запомните data.id.
PUT .../warehouses/{id}	Изменить код/название склада	region + warehouse_id из GET. Body: {"name":"Новое имя"} (code можно не трогать).
DELETE .../warehouses/{id}	Удалить склад	region + id. Не удаляйте склад, если на нём есть остатки/заявки (БД может отказать по FK).

6.0.3. Товары (products)

Кнопка / метод	Что делает	Как пользоваться
GET .../products	Список номенклатуры на шарде региона	region → Execute. Справочник копируется на оба региона (реплика справочника).
POST .../products	Добавить товар	Body: {"sku":"SKU-400","title":"Кабель 3м"}. sku должен быть уникальным.
PUT .../products/{id}	Изменить sku/title	product_id из GET. Body: {"title":"Новое название"}.
DELETE .../products/{id}	Удалить товар	Только если на него нет ссылок в stock/items.

6.0.4. Остатки (stock)

Кнопка / метод	Что делает	Как пользоваться
GET .../stock	Остатки складов региона (available / reserved)	region → Execute. Смотрите qty_available и qty_reserved.
POST .../stock	Завести остаток товара на складе	Body: {"warehouse_id":1,"product_id":1,"qty_available":40}. id склада и товара — из предыдущих GET/POST.
PUT .../stock/{id}	Изменить доступное количество	stock_id из GET. Body: {"qty_available":50}. Резерв меняется сценарием статусов, не вручную.
DELETE .../stock/{id}	Удалить строку остатка	region + stock_id. Используйте осторожно на демо-данных.

6.0.5. Заявки на отгрузку (orders) — главный сценарий

Кнопка / метод	Что делает	Как пользоваться
GET .../orders	Список заявок региона	region=WEST → Execute. Видны id и status (CREATED, RESERVED, ...).
POST .../orders	Создать заявку + позицию (статус CREATED) и первую запись в events	Body: {"warehouse_id":1,"product_id":1,"qty":3}. Склад должен быть этого же региона. Запомните data.order.id.
POST .../orders/{id}/status	Сменить статус (предметный сценарий сверх CRUD)	order_id + Body: {"to_status":"RESERVED","note":"резерв"}. Дальше по цепочке: PICKING → SHIPPED → DELIVERED. Или CANCELLED из CREATED/RESERVED.
GET .../orders/{id}/events	Журнал смен статусов заявки (чтение с replica)	Тот же region и order_id → Execute. Каждая смена = строка from_status → to_status.
POST /api/cross-region-denied	Демонстрация отказа по критерию контура (без реальной записи)	Query: from_region=WEST, to_region=EAST, order_id=<id заявки WEST>. Execute → всегда 403 с пояснением правила периметра. Альтернатива: POST /api/EAST/orders/{WEST_id}/status.

6.0.6. Рекомендуемый порядок нажатий «с нуля»

- 1) GET /health — все узлы ok
- 2) POST /api/WEST/warehouses — создать склад, взять id
- 3) POST /api/WEST/products — создать товар, взять id
- 4) POST /api/WEST/stock — остаток (warehouse_id + product_id)
- 5) POST /api/WEST/orders — заявка qty=3 → order.id
- 6) POST .../orders/{id}/status — RESERVED
- 7) POST .../status — PICKING → SHIPPED → DELIVERED
- 8) GET .../orders/{id}/events — журнал контроля
- 9) POST /api/EAST/orders/{WEST_id}/status — должен вернуть 403 (чужой регион)

Допустимые переходы статусов: CREATED→RESERVED|CANCELLED; RESERVED→PICKING|CANCELLED; PICKING→SHIPPED; SHIPPED→DELIVERED. Любой другой переход Swagger покажет ошибку 400.

6.1. CRUD ключевых сущностей (сводка)

Сущность	Create	Read	Update	Delete
warehouses	POST	GET (replica)	PUT	DELETE

Сущность	Create	Read	Update	Delete
products	POST	GET	PUT	DELETE
stock	POST	GET	PUT	DELETE
orders	POST	GET	через status	CANCELLED (отмена)

6.2. Предметный сценарий: жизненный цикл заявки (статусы)

Это главный сценарий сверх обычного CRUD: заявка на отгрузку проходит цепочку состояний. Статус — не «просто строка в интерфейсе», а правило склада: когда товар резервируется, когда списывается, когда отгрузка завершена или отменена.

6.2.1. Где это лежит

Что	Где физически	Как меняется
Текущий статус заявки	Таблица shipment_orders, поле status (на primary/replica своего региона)	UPDATE при каждом успешном POST .../status
История всех шагов	Таблица shipment_events (журнал) на том же фрагменте	INSERT новой строки на каждый переход
Остатки под резерв	Таблица stock_balances: qty_available / qty_reserved	Меняются при RESERVED / CANCELLED / SHIPPED
Логика переходов	Клиент client/app/main.py: словарь TRANSITIONS + функции _reserve/_ship/_unreserve	API: POST /api/{region}/orders/{id}/status
Ограничение в БД	CHECK status IN (CREATED, RESERVED, PICKING, SHIPPED, DELIVERED, CANCELLED)	Невалидный статус БД не примет

6.2.2. Что означает каждый статус

Сначала три слова, без которых таблица непонятна:

- qty — сколько штук товара заказали в заявке (поле в shipment_items). Пример: заявка на 3 кабеля → qty = 3.
- qty_available (available) — сколько сейчас свободно на складе, можно отдать под новые заявки (поле в stock_balances).
- qty_reserved (reserved) — сколько уже «забронировано» под существующие заявки и пока нельзя отдать другим.

Числовой пример. На складе было available = 10, reserved = 0. Создали заявку qty = 3. При переходе в RESERVED: available становится 7, reserved становится 3. Если попытаться зарезервировать qty = 15 при available = 10 — отказ: свободного товара не хватает.

Статус	По-русски	Смысл для склада	Что с остатком (простыми словами)
CREATED	создана	Заявку оформили, товар ещё не держат	Ничего: available и reserved как были
RESERVED	зарезервирована	Забронировали qty штук под эту заявку	Свободных меньше на qty, в резерве больше на qty. Нельзя, если свободных меньше, чем qty
PICKING	сборка	Собирают заказ; бронь ещё держится	Цифры остатка не трогаем
SHIPPED	отгружена	Товар уехал со склада	Из резерва убираем qty (товар уже не на складе)
DELIVERED	доставлена	Успешный конец; дальше статус не меняют	Цифры остатка не трогаем
CANCELLED	отменена	Отменили до отгрузки	Если успели зарезервировать — вернуть qty обратно в свободные

6.2.3. CANCELLED — отдельно (чтобы объяснить на защите)

CANCELLED — это не «удалить заявку из таблицы», а статус отмены: наряд остаётся в БД (для аудита), но жизненный цикл закончен неудачей/отменой. После CANCELLED переходов дальше нет (как и после DELIVERED).

- Отменить можно только из CREATED или RESERVED (пока товар не ушёл в сборку/отгрузку).
- Из PICKING / SHIPPED / DELIVERED в CANCELLED перейти нельзя — клиент вернёт ошибку 400 «переход запрещён».
- Если отмена из RESERVED: резерв снимается (qty_reserved –, qty_available +) — товар снова свободен.
- Если отмена из CREATED: остатки не трогали, меняется только status + строка в журнале.

- В журнале появится, например: CREATED→CANCELLED или RESERVED→CANCELLED.

6.2.4. Какие переходы разрешены (логика автомата)



Рисунок 3 — UML state: жизненный цикл shipment_orders.status

```

CREATED → RESERVED | CANCELLED
RESERVED → PICKING | CANCELLED
PICKING → SHIPPED
SHIPPED → DELIVERED
DELIVERED → (конец)
CANCELLED → (конец)
  
```

Успешный путь: CREATED → RESERVED → PICKING → SHIPPED → DELIVERED. Боковой путь отмены: ... → CANCELLED. Реализовано словарём TRANSITIONS в client/app/main.py: перед UPDATE клиент проверяет, есть ли to_status в множестве допустимых из текущего статуса.

6.2.5. Как это вызывается и что происходит по шагам

Эндпоинт: POST /api/{region}/orders/{order_id}/status с телом {"to_status": "RESERVED", "note": "..."}.

1. Найти заявку FOR UPDATE на фрагменте region.
2. Проверить order.region_code == region, иначе 403 (периметр).
3. Проверить переход по TRANSITIONS, иначе 400.
4. При необходимости изменить остатки (_reserve / _unreserve / _ship).
5. UPDATE shipment_orders.status = новый статус.
6. INSERT в shipment_events (from_status, to_status, note) — фиксация контроля.
7. Async replication копирует изменения на replica региона.

Чужой регион → HTTP 403. Нельзя зарезервировать больше available → ошибка при RESERVED. Это и есть предметный сценарий задания: не просто INSERT/SELECT, а цепочка состояний + контроль.

6.3. Шаги demo.sh (полный прогон API)

- health; CRUD warehouses/products/stock на WEST и EAST (list/create/update);
- заявка WEST → цепочка до DELIVERED; заявка EAST → до SHIPPED; чтение events;
- CANCELLED из CREATED и из RESERVED (с проверкой возврата остатка);
- негативы: запрещённый переход (400), отмена из PICKING (400), нехватка остатка (409);
- критерий контура: статус/events чужого региона + /api/cross-region-denied → 403;
- docker stop west-primary → отказ записи WEST, чтение с replica, EAST жив; docker start → запись снова работает;
- опционально CLEANUP=1 — удаление демо stock/warehouse/product.

7. Код запросов (фрагменты)

7.1. Создание заявки (запись → primary)

```
INSERT INTO shipment_orders (warehouse_id, region_code, status)
VALUES (%s, %s, 'CREATED') RETURNING *;
INSERT INTO shipment_items (order_id, product_id, qty)
VALUES (%s, %s, %s);
INSERT INTO shipment_events (order_id, from_status, to_status, note)
VALUES (%s, NULL, 'CREATED', 'order created');
```

7.2. Смена статуса + журнал (контроль)

```
SELECT * FROM shipment_orders WHERE id = %s FOR UPDATE;
-- проверка region_code == запрошенный регион, иначе 403
-- RESERVED: qty_available -= qty; qty_reserved += qty
-- SHIPPED: qty_reserved -= qty
UPDATE shipment_orders SET status = %s, updated_at = NOW() WHERE id = %s;
INSERT INTO shipment_events (order_id, from_status, to_status, note)
VALUES (%s, %s, %s, %s);
```

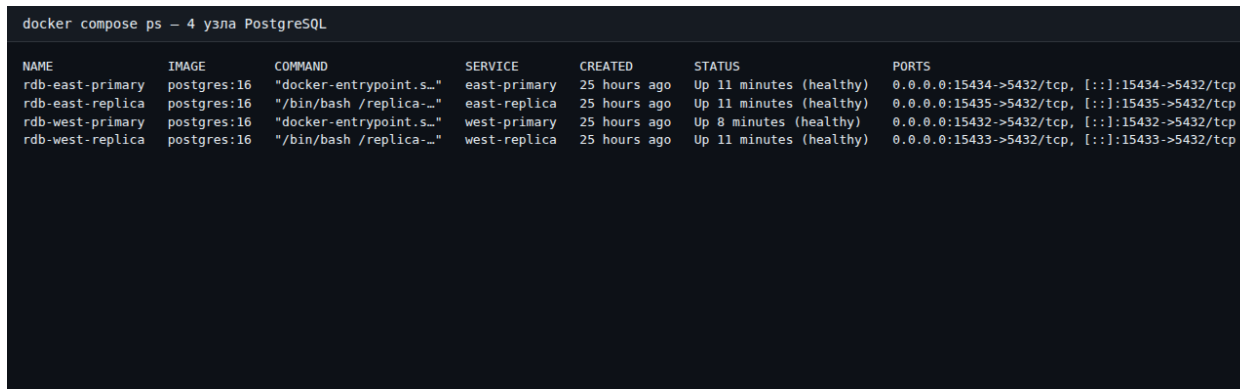
7.3. Маршрутизация клиента

запись: connect(region, 'primary')
чтение: сначала replica, при недоступности — primary
ответ: { routing: { region, role, host, criterion }, data: ... }

8. Результаты выполнения (скриншоты)

Снимки живого стенда после прогона scripts/demo.sh и scripts/capture_screenshots.py.

8.1. Стенд: docker compose ps — 4 узла PostgreSQL



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
rdb-east-primary	postgres:16	"docker-entrypoint.s..."	east-primary	25 hours ago	Up 11 minutes (healthy)	0.0.0.0:15434->5432/tcp, [::]:15434->5432/tcp
rdb-east-replica	postgres:16	"/bin/bash /replica-..."	east-replica	25 hours ago	Up 11 minutes (healthy)	0.0.0.0:15435->5432/tcp, [::]:15435->5432/tcp
rdb-west-primary	postgres:16	"docker-entrypoint.s..."	west-primary	25 hours ago	Up 8 minutes (healthy)	0.0.0.0:15432->5432/tcp, [::]:15432->5432/tcp
rdb-west-replica	postgres:16	"/bin/bash /replica-..."	west-replica	25 hours ago	Up 11 minutes (healthy)	0.0.0.0:15433->5432/tcp, [::]:15433->5432/tcp

Рисунок 8.1 — 00_docker_compose_ps.png

8.2. Swagger UI: список API

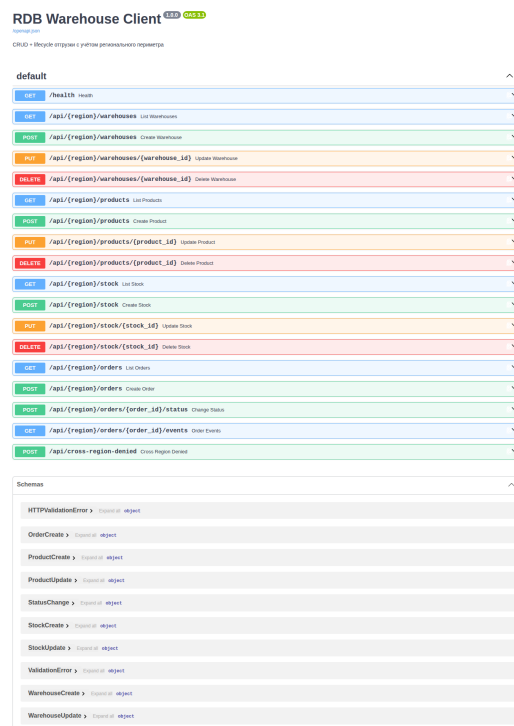


Рисунок 8.2 — 01_swagger_home.png

8. Результаты выполнения (продолжение)

8.3. CRUD: создание склада → primary WEST

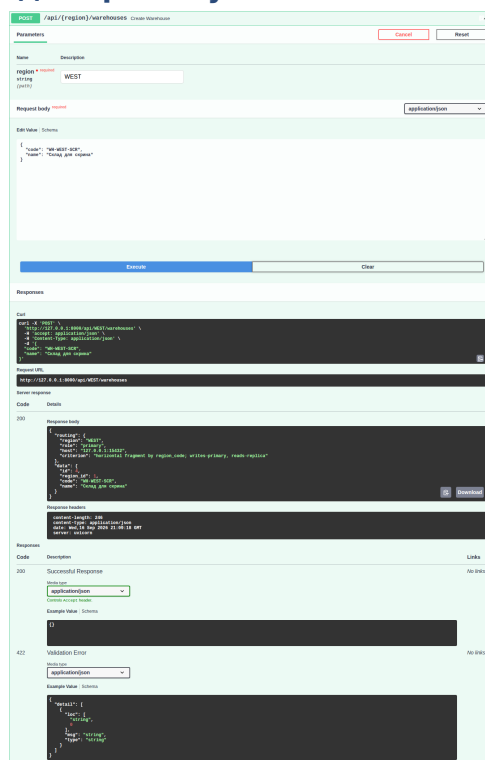


Рисунок 8.3 — 05_swagger_warehouse_post.png

8.4. CRUD: создание товара → primary WEST

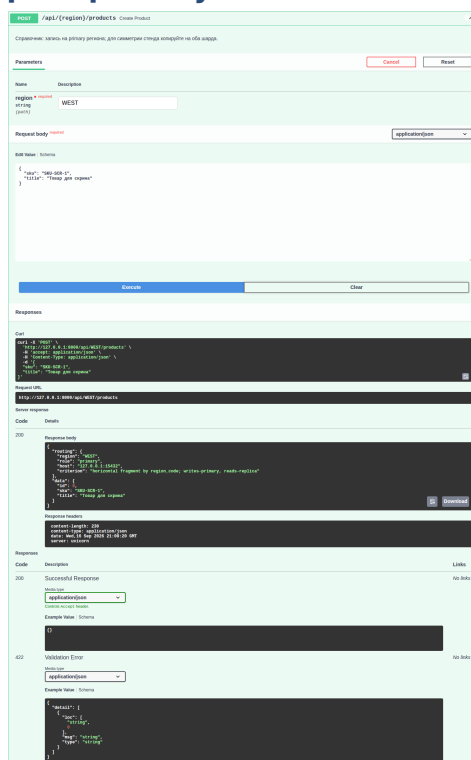


Рисунок 8.4 — 06_swagger_product_post.png

8. Результаты выполнения (продолжение)

8.5. CRUD: создание остатка

```
POST /api/WEST/stock – создание остатка

{
  "routing": {
    "region": "WEST",
    "role": "primary",
    "host": "127.0.0.1:15432",
    "criterion": "horizontal fragment by region_code; writes-primary, reads-replica"
  },
  "data": {
    "id": 7,
    "warehouse_id": 3,
    "product_id": 7,
    "qty_available": 25,
    "qty_reserved": 0
  }
}
```

Рисунок 8.5 — api_07_stock_create.png

8.6. Создание заявки на отгрузку

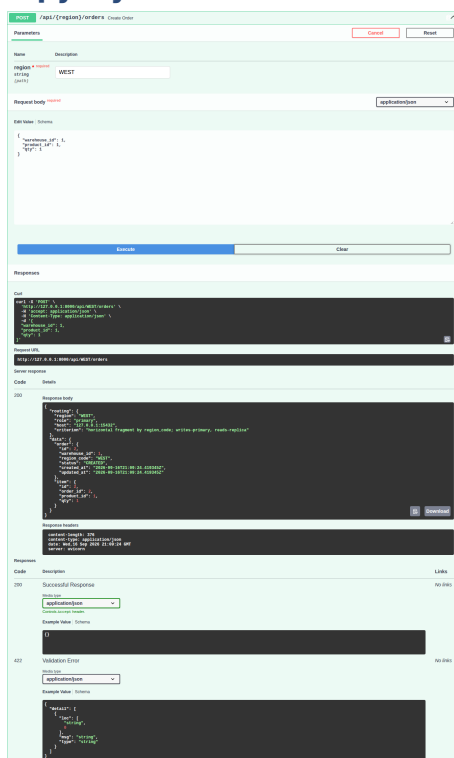


Рисунок 8.6 — 08_swagger_order_post.png

8. Результаты выполнения (продолжение)

8.7. Цепочка статусов → DELIVERED

```
Цепочка статусов → DELIVERED

{
  "routing": {
    "region": "WEST",
    "role": "primary",
    "host": "127.0.0.1:15432",
    "criterion": "horizontal fragment by region_code; writes-primary, reads-replica"
  },
  "data": {
    "order": {
      "id": 1,
      "warehouse_id": 3,
      "region_code": "WEST",
      "status": "DELIVERED",
      "created_at": "2026-09-16T21:02:00.482169Z",
      "updated_at": "2026-09-16T21:02:00.670150Z"
    },
    "event": {
      "id": 5,
      "order_id": 1,
      "from_status": "SHIPPED",
      "to_status": "DELIVERED",
      "created_at": "2026-09-16T21:02:00.670150Z",
      "note": "demo DELIVERED"
    }
  }
}
```

Рисунок 8.7 — api_10_status_DELIVERED.png

8.8. Журнал shipment_events

```
GET events — журнал shipment_events

{
  "routing": {
    "region": "WEST",
    "role": "replica",
    "host": "127.0.0.1:15433",
    "criterion": "horizontal fragment by region_code; writes-primary, reads-replica"
  },
  "data": [
    {
      "id": 1,
      "order_id": 1,
      "from_status": null,
      "to_status": "CREATED",
      "created_at": "2026-09-16T21:02:00.482169Z",
      "note": "order created"
    },
    {
      "id": 2,
      "order_id": 1,
      "from_status": "CREATED",
      "to_status": "RESERVED",
      "created_at": "2026-09-16T21:02:00.560929Z",
      "note": "demo RESERVED"
    },
    {
      "id": 3,
      "order_id": 1,
      "from_status": "RESERVED",
      "to_status": "PICKING",
      "created_at": "2026-09-16T21:02:00.595572Z",
      "note": "demo PICKING"
    },
    {
      "id": 4,
      "order_id": 1,
      "from_status": "PICKING",
      "to_status": "SHIPPED",
      "created_at": "2026-09-16T21:02:00.631543Z",
      "note": "demo SHIPPED"
    },
    {
      "id": 5,
      "order_id": 1,
      "from_status": "SHIPPED",
      "to_status": "DELIVERED",
      "created_at": "2026-09-16T21:02:00.670150Z",
      "note": "demo DELIVERED"
    }
  ]
}
```

Рисунок 8.8 — api_11_events.png

8. Результаты выполнения (продолжение)

8.9. Отказ по критерию контура (чужой регион)

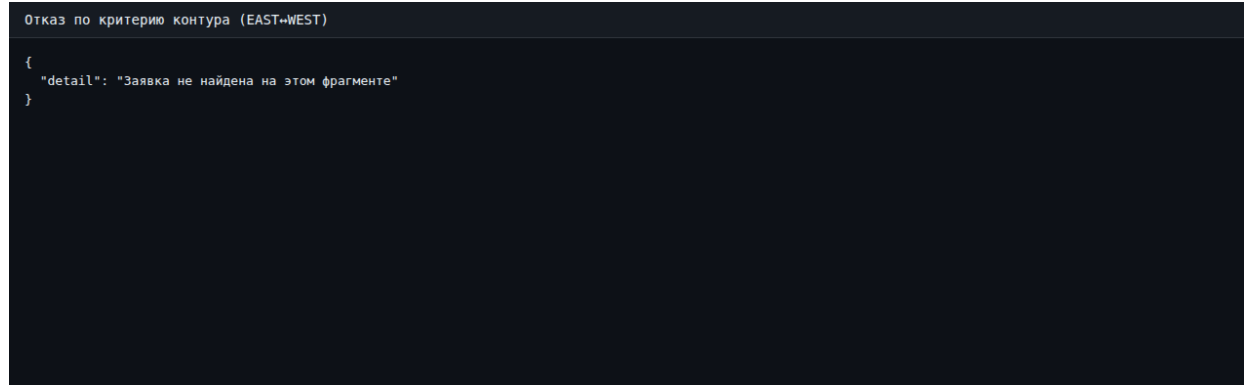


Рисунок 8.9 — api_12_cross_region.png

8.10. Отказ записи при docker stop west-primary



Рисунок 8.10 — api_13_write_fail.png

8. Результаты выполнения (продолжение)

8.11. Чтение с replica при недоступном primary

```
Чтение с replica при отказе primary

{
  "routing": {
    "region": "WEST",
    "role": "replica",
    "host": "127.0.0.1:15433",
    "criterion": "horizontal fragment by region_code; writes-primary, reads-replica"
  },
  "data": [
    {
      "id": 1,
      "warehouse": "WH-WEST-1",
      "sku": "SKU-100",
      "title": "Кабель оптический 1м",
      "qty_available": 100,
      "qty_reserved": 0
    },
    {
      "id": 2,
      "warehouse": "WH-WEST-1",
      "sku": "SKU-200",
      "title": "Блок питания ИБП",
      "qty_available": 50,
      "qty_reserved": 0
    },
    {
      "id": 3,
      "warehouse": "WH-WEST-1",
      "sku": "SKU-300",
      "title": "Коммутатор 24п",
      "qty_available": 20,
      "qty_reserved": 0
    },
    {
      "id": 7,
      "warehouse": "WH-WEST-DEMO",
      "sku": "SKU-DEMO-1",
      "title": "Демо-товар стенда (upd)",
      "qty_available": 22,
      "qty_reserved": 0
    }
  ]
}
```

Рисунок 8.11 — api_13_read_replica.png

8.12. Восстановление после docker start primary

```
Health после docker start west-primary

{
  "checked_at": "2026-09-16T21:02:06.320720+00:00",
  "nodes": [
    {
      "region": "WEST",
      "role": "primary",
      "dsn_host": "127.0.0.1:15432",
      "ok": true,
      "detail": "in_recovery=False"
    },
    {
      "region": "WEST",
      "role": "replica",
      "dsn_host": "127.0.0.1:15433",
      "ok": true,
      "detail": "in_recovery=True"
    },
    {
      "region": "EAST",
      "role": "primary",
      "dsn_host": "127.0.0.1:15434",
      "ok": true,
      "detail": "in_recovery=False"
    },
    {
      "region": "EAST",
      "role": "replica",
      "dsn_host": "127.0.0.1:15435",
      "ok": true,
      "detail": "in_recovery=True"
    }
  ]
}
```

Рисунок 8.12 — api_14_health_after_restart.png

9. Электронные материалы

```
Индивидуальный_проект/  
├── docker-compose.yml  
├── db/init/01_schema.sql, west|east/02_seed.sql  
├── dump/west_warehouse.sql, east_warehouse.sql ← выгрузка pg_dump  
├── db/primary/, db/replica/  
├── client/app/main.py, requirements.txt  
├── scripts/bootstrap_schema.sh, demo.sh, make_dump.sh  
├── screenshots/  
├── docs/Заявка_....pdf  
└── Отчёт_индивидуальный_проект_РБД.pdf ← этот файл
```

Выгрузка базы: каталог dump/ (west_warehouse.sql, east_warehouse.sql, архив tar.gz).
Переснять: ./scripts/make_dump.sh. Секреты подключений учебные (warehouse/warehouse),
только для локального Docker.

10. Заключение

Реализован распределённый учебный стенд с горизонтальной фрагментацией по region_code, асинхронными репликами на чтение, клиентом FastAPI (CRUD + цепочка статусов отгрузки + журнал событий) и демонстрацией отказов (узел primary, критерий контура). Критерий распределения обоснован периметром площадки и режимом доступа, а не балансировкой нагрузки.